

Blogging Platform

Database Fundamentals

Complexity: Advanced | **Time Estimate:** 10–12 hours

Project Objectives

By the end of this project, learners will be able to:

- Design and normalize a relational or non-relational database schema that models a blogging domain effectively.
 - Develop conceptual, logical, and physical database models to ensure scalability and maintainability.
 - Implement CRUD operations and complex queries using SQL (or equivalent NoSQL queries) and JDBC.
 - Apply indexing, caching, searching, and sorting algorithms to optimize database access and performance.
 - Integrate database operations into a JavaFX application interface for interactive content management.
 - Compare relational (SQL) and NoSQL designs for unstructured data storage such as comments or reviews.
 - Measure and document performance improvement through optimization and indexing techniques.
-

Project Description

This stage of the **Smart Blogging Platform** focuses on building the **data layer and persistence logic** that will support all future modules.

Students will design a relational or non-relational database for a blogging system, implement the schema using SQL or NoSQL, and integrate it with a **JavaFX application** that performs core operations, including post creation, comment management, tag assignment, and analytics reporting.

The system must reflect real-world blogging logic (**users, posts, comments, tags, and reviews**) while incorporating **data structure and algorithm (DSA)** concepts such as **indexing, caching, sorting, and searching** to enhance performance and scalability.

Epics and User Stories

Epic 1: Database Design and Modeling

User Story 1.1:

As a database designer, I want to create conceptual, logical, and physical database models so that the data structure of the blogging platform is efficient and scalable.

Acceptance Criteria:

- Conceptual ERD or NoSQL schema includes all major entities and relationships.
- Logical model defines attributes, primary/foreign keys, and relationships.
- Physical model includes SQL data types, constraints, and normalization (up to 3NF where applicable).

User Story 1.2:

As a database administrator, I want to define indexes and relationships so that queries and lookups run efficiently while maintaining data integrity.

Acceptance Criteria:

- Appropriate primary and foreign keys (or references for NoSQL) are defined.
 - Indexes are created on frequently queried columns or document fields (e.g., post title, author ID, tag name).
 - Referential integrity and constraints are enforced.
-

Epic 2: Data Access and CRUD Operations

User Story 2.1:

As a blogger, I want to add, update, and delete blog posts and comments through a JavaFX interface so that I can manage my content easily.

Acceptance Criteria:

- All CRUD operations functional through the JavaFX interface.
- Input validation and user feedback messages implemented.
- Database constraints or schema validation prevent duplicate or invalid entries.

User Story 2.2:

As a reader, I want to view and search for blog posts and their details so that I can explore content of interest.

Acceptance Criteria:

- Post listings displayed dynamically from the database.
 - Pagination, filters, or search functions implemented for improved usability.
 - Data retrieved using secure parameterized queries through JDBC or NoSQL driver.
-

Epic 3: Searching, Sorting, and Optimization

User Story 3.1:

As a reader, I want to search for posts quickly by keyword, tag, or author so that I can find relevant articles efficiently.

Acceptance Criteria:

- Post search is case-insensitive and responsive.
- Search performance improved through indexing or hashing.
- Measurable improvement in query execution time documented.

User Story 3.2:

As a developer, I want to implement caching and sorting so that frequently accessed posts are retrieved faster.

Acceptance Criteria:

- Caching layer implemented using in-memory data structures (e.g., Maps or Lists).
 - Sorting and searching algorithms applied to cached data.
 - Cache invalidation ensures consistent results after updates.
-

Epic 4: Performance and Query Optimization

User Story 4.1:

As an analyst, I want to generate performance reports comparing pre- and post-optimization queries so that I can measure system efficiency.

Acceptance Criteria:

- Query execution times recorded before and after applying indexes or caching.

- Indexes and query optimizations demonstrate measurable performance gains.
- Performance report clearly explains methodology and findings.

User Story 4.2:

As a developer, I want to store user comments or reviews in a NoSQL format so that unstructured feedback can be handled flexibly.

Acceptance Criteria:

- NoSQL data model created for reviews or user comments.
 - Justification for using NoSQL documented (e.g., scalability, flexibility).
 - Optional implementation comparison included in performance report.
-

Epic 5: Reporting and Documentation

User Story 5.1:

As a project contributor, I want to document the database schema and project steps so that the system can be extended or maintained easily.

Acceptance Criteria:

- ERD diagrams or NoSQL schemas documented clearly.
 - SQL/NoSQL scripts and test data provided.
 - README file explains setup, dependencies, and execution instructions.
-

Technical Requirements

Database

- Use **MySQL** or **PostgreSQL** for the relational implementation or **MongoDB** for NoSQL.
- Schema normalized to **Third Normal Form (3NF)** for SQL databases.
- Include at least these entity groups: **Users, Posts, Comments, Tags, Reviews**.
- Define indexes on frequently queried columns (e.g., post title, tag name, author ID).
- Enforce referential integrity with foreign keys or document references.

Application Layer (JavaFX + JDBC / NoSQL Driver)

- Use **JavaFX** for user interaction: displaying posts, searching, CRUD operations, and viewing analytics.
- Database access implemented using **JDBC** (SQL) or **MongoDB Java Driver** (NoSQL) with parameterized queries to prevent injection.
- Apply layered design: **Controller** → **Service** → **DAO**.
- Use in-memory data structures (e.g., Maps, Lists) for caching and local sorting.
- Include a dashboard for managing posts, viewing performance metrics, and performing optimizations.

Data Structures & Algorithms Integration

- **Hashing / Caching:** Use in-memory structures to store frequently accessed posts and comments.
 - **Sorting & Searching:** Apply algorithms (e.g., QuickSort, Binary Search) to optimize query results.
 - **Indexing Concept:** Explain how database indexing relates to in-memory search and sorting.
 - **Performance Measurement:** Compare query response times before and after optimization.
-

Deliverables

	≡ DELIVERABLE	≡ DESCRIPTION
1	Database Design Document	Conceptual, logical, and physical models with ERDs or NoSQL schema diagrams and explanations.
2	SQL / NoSQL Implementation Script	Scripts for creating tables/collections, constraints, indexes, and inserting sample data.
3	JavaFX Application	Interactive interface performing CRUD, search, and reporting functions using JDBC or NoSQL driver.
4	Performance Report	Comparative analysis showing pre- and post-optimization metrics (indexing, caching, and query optimization).
5	NoSQL Design (Optional)	Documented schema or partial implementation for storing unstructured data such as comments or logs.
6	README File	Setup guide including environment configuration, dependencies, database connection setup, and execution steps.
7	Testing Evidence	Screenshots or console logs showing correct query results and validation outputs.

Evaluation Criteria

	≡ CATEGORY	≡ EVALUATION CRITERIA	≡ POINTS
1	Database Design (25 pts)	Conceptual, logical, and physical models are complete, normalized (3NF), and well-documented with ERDs or schema diagrams.	25
2	SQL / NoSQL Implementation (20 pts)	Schema or collections are syntactically correct, include constraints, indexes, and support complex queries.	20
3	JavaFX + JDBC Integration (20 pts)	CRUD functionality, UI usability, and safe JDBC/driver handling correctly implemented with clear separation of concerns.	20
4	DSA Application (15 pts)	Caching, sorting, and searching techniques applied with performance justification and accurate mapping to indexing concepts.	15

5	Performance Optimization (10 pts)	Measurable improvements demonstrated through indexing, caching, and documented analysis.	10
6	Documentation & Code Quality (10 pts)	README completeness, logical organization, code clarity, and adherence to clean coding standards.	10
7	Total		100 pts